

The Right-Left Rule for C Declarations

The “right-left” rule is a completely regular rule for deciphering C declarations. It can also be useful in creating them.

First, symbols. Read

*	as "pointer to"	- always on the left side
[]	as "array of"	- always on the right side
()	as "function returning"	- always on the right side

as you encounter them in the declaration.

Step 1

Find the identifier. This is your starting point. Then say to yourself, “identifier is.” You’ve started your declaration.

Step 2

Look at the symbols on the right of the identifier. If, say, you find “()” there, then you know that this is the declaration for a function. So you would then have “identifier is function returning”. Or if you found a “[]” there, you would say “identifier is array of”. Continue right until you run out of symbols *OR* hit a *right* parenthesis “)”. (If you hit a left parenthesis, that’s the beginning of a () symbol, even if there is stuff in between the parentheses. More on that below.)

Step 3

Look at the symbols to the left of the identifier. If it is not one of our symbols above (say, something like “int”), just say it. Otherwise, translate it into English using that table above. Keep going left until you run out of symbols *OR* hit a *left* parenthesis “(”.

Now repeat steps 2 and 3 until you’ve formed your declaration. Here are some examples:

```
int *p[];
```

1. Find identifier.

```
int *p[];
```

“p is”

2. Move right until out of symbols or right parenthesis hit.

```
int *p[];  
    ^^
```

“p is array of”

3. Can’t move right anymore (out of symbols), so move left and find:

```
int *p[];  
    ^
```

“p is array of pointer to”

4. Keep going left and find:

```
int *p[];  
    ^^^
```

“p is array of pointer to int”. (or “p is an array where each element is of type pointer to int”)

Another example:

```
int *(*func())();
```

1. Find the identifier.

```
int *(*func())();  
    ^^^^^
```

“func is”

2. Move right.

```
int *(*func())();  
    ^^
```

“func is function returning”

3. Can’t move right anymore because of the right parenthesis, so move left.

```
int *(*func())();  
    ^
```

“func is function returning pointer to”

4. Can't move left anymore because of the left parenthesis, so keep going right.

```
int *(*func())();  
      ^^
```

“func is function returning pointer to function returning”

5. Can't move right anymore because we're out of symbols, so go left.

```
int *(*func())();  
      ^
```

“func is function returning pointer to function returning pointer to”

6. And finally, keep going left, because there's nothing left on the right.

```
int *(*func())();  
^^^^
```

“func is function returning pointer to function returning pointer to int”.

As you can see, this rule can be quite useful. You can also use it to sanity check yourself while you are creating declarations, and to give you a hint about where to put the next symbol and whether parentheses are required.

Some declarations look much more complicated than they are due to array sizes and argument lists in prototype form. If you see “[3]”, that's read as “array (size 3) of...”. If you see “(char *,int)” that's read as “function expecting (char *,int) and returning...”. Here's a fun one:

```
int ((*fun_one)(char *,double))[9][20];
```

I won't go through each of the steps to decipher this one.

Ok. It's:

```
"fun_one is pointer to function expecting (char *,double) and  
returning pointer to array (size 9) of array (size 20) of int."
```

As you can see, it's not as complicated if you get rid of the array sizes and argument lists:

```
int ((*fun_one)())[] [];
```

You can decipher it that way, and then put in the array sizes and argument lists later.

Some final words:

It is quite possible to make illegal declarations using this rule, so some knowledge of what's legal in C is necessary. For instance, if the above had been:

```
int *((*fun_one)())[] [];
```

it would have been “fun_one is pointer to function returning array of array of pointer to int”. Since a function cannot return an array, but only a pointer to an array, that declaration is illegal.

Illegal combinations include:

```
[]() - cannot have an array of functions
]() - cannot have a function that returns a function
()[] - cannot have a function that returns an array
```

In all the above cases, you would need a set of parens to bind a * symbol on the left between these () and [] right-side symbols in order for the declaration to be legal.

Here are some legal and illegal examples:

int i;	an int
int *p;	an int pointer (ptr to an int)
int a[];	an array of ints
int f();	a function returning an int
int **pp;	a pointer to an int pointer (ptr to a ptr to an int)
int (*pa)[];	a pointer to an array of ints
int (*pf)();	a pointer to a function returning an int
int *ap[];	an array of int pointers (array of ptrs to ints)
int aa[] [];	an array of arrays of ints
int af[] ();	an array of functions returning an int (ILLEGAL)
int *fp();	a function returning an int pointer
int fa() [];	a function returning an array of ints (ILLEGAL)
int ff() ();	a function returning a function returning an int (ILLEGAL)
int ***ppp;	a pointer to a pointer to an int pointer
int (**ppa)[];	a pointer to a pointer to an array of ints
int (**ppf)();	a pointer to a pointer to a function returning an int
int *(*pap)[];	a pointer to an array of int pointers
int (*paa)[] [];	a pointer to an array of arrays of ints
int (*paf)[] ();	a pointer to a an array of functions returning an int (ILLEGAL)
int *(*pfp)();	a pointer to a function returning an int pointer
int (*pfa)() [];	a pointer to a function returning an array of ints (ILLEGAL)
int (*pff)() ();	a pointer to a function returning a function returning an int (ILLEGAL)
int **app[];	an array of pointers to int pointers
int (*apa[])[];	an array of pointers to arrays of ints
int (*apf[])();	an array of pointers to functions returning an int
int *aap[] [];	an array of arrays of int pointers

<code>int aaa[] [] [];</code>	an array of arrays of arrays of ints
<code>int aaf[] [] ();</code>	an array of arrays of functions returning an int (ILLEGAL)
<code>int *afp[] ();</code>	an array of functions returning int pointers (ILLEGAL)
<code>int afa[] () [];</code>	an array of functions returning an array of ints (ILLEGAL)
<code>int aff[] () ();</code>	an array of functions returning functions returning an int (ILLEGAL)
<code>int **fpp();</code>	a function returning a pointer to an int pointer
<code>int (*fpa()) [];</code>	a function returning a pointer to an array of ints
<code>int (*fpf()) ();</code>	a function returning a pointer to a function returning an int
<code>int *fap() [];</code>	a function returning an array of int pointers (ILLEGAL)
<code>int faa() [] [];</code>	a function returning an array of arrays of ints (ILLEGAL)
<code>int faf() [] ();</code>	a function returning an array of functions returning an int (ILLEGAL)
<code>int *ffp() ();</code>	a function returning a function returning an int pointer (ILLEGAL)